Experimental Setups, Comparative Benchmarks, and Codebase Verification

Post-Quantum Cryptography & SecureMark-TLS Profiles on Embedded Microcontrollers

---

1 Experimental Environments and Hardware Measurement Setup

To evaluate post-quantum cryptographic workloads against classical TLS primitives on resource-constrained edge hardware, two target microcontrollers representing modern Cortex-M and Xtensa architectures were configured:

- **NXP FRDM-MCXN947 Target:** Dual ARM Cortex-M33 cores operating at 150 MHz (ARMv8-M Mainline with TrustZone and DSP extensions), equipped with 2 MB dual-bank Flash, 512 KB SRAM, and an on-chip eIQ Neutron NPU with dedicated DMA bus routing. Toolchain: `arm-none-eabi-gcc` v13.2.1 with flags `-O2` (speed) and `-Os` (`SIZE_OPTIMIZATIONS`), `-mcpu=cortex-m33+nodsp`, `-mfpu=fpv5-sp-d16`, `-mfloat-abi=hard`. Operating environment: Zephyr RTOS v4.2.0-rc1.
- **ESP32-S3 DevKitC Target:** Dual Xtensa LX7 cores operating at 240 MHz, with 8 MB Octal SPI Flash and 512 KB SRAM. Toolchain: `xtensa-esp32s3-elf-gcc` v12.2.0 with flags `-Os` and comparative `-O2`. Operating environment: ESP-IDF v5.1 / Zephyr RTOS v4.2.0-rc1.

1.1 Instrumentation Methodology

- **Execution Cycles & Latency:** Measured directly via internal hardware registers using the ARM Data Watchpoint and Trace (DWT) `CYCCNT` cycle counter on the MCXN947 and the Xtensa `CCOUNT` register on the ESP32-S3. All tests were executed in free-running mode with debuggers and UART streams detached to eliminate clock jitter.
- **Dynamic Stack Memory High-Water Mark (HWM):** Stack memory regions were pre-painted at boot with a distinct byte pattern (`0xEE`). Following execution, stack memory boundaries were scanned using `k_thread`$_{s}$$tack_space_gettorecordtheexactpeakbyteusage.$`Active Power & Energy Profiling:`$Capturedusing a NordicPo$
- **Sample Size:** 100 repetitions per KEM operation and 2,000 repetitions per DSA operation, yielding standard deviations under 0.5%.

2 Bare-Metal EEMBC SecureMark-TLS Classical Profile

The classical baseline was established using the self-hosted EEMBC SecureMark-TLS benchmark suite, running mbedTLS (v3.6.3 on MCXN947; ESP-IDF mbedTLS on ESP32-S3) across 14 cryptographic phases modeling a TLS 1.2 handshake (ECDH P-256, ECDSA P-256, AES-128-CBC/CCM, SHA-256).

- **NXP FRDM-MCXN947 Score (`-O2`): 5,472.08 SecureMark Marks** (Passed 14/14 CRC component validations; file: `securemark_score.txt`).
- **ESP32-S3 DevKitC Score (`-Os`): 1,895.06 SecureMark Marks** (Passed 14/14 CRC component validations; file: `securemark_esp32s3_score.txt`).

2.1 Low-Level Porting Adjustments

1. **ECP Point Blinding PRNG:** Implemented a lightweight `xorshift` pseudo-random generator to feed mbedTLS point blinding, preventing null-pointer `MBEDTLS_ERR_ECP_BAD_INPUT_DATA` aborts.
2. **SEC1 Public Key Header Formatting:** Dynamically prepended the SEC1 `0x04` byte to raw 64-byte $X||Y$ EEMBC public key arrays before passing them to mbedTLS parsing functions.
3. **ASN.1 DER Signature Formatting:** Formatted raw $r||s$ hardware signature registers into standard DER structures required for EEMBC CRC verification.

3 Post-Quantum Primitive Benchmarks (EEMBC-Style Profile)

Table 1 summarizes the bare-metal execution cycles, latencies, dynamic stack RAM HWM, and binary code sizes for FIPS-203 ML-KEM and FIPS-204 ML-DSA primitives on the NXP FRDM-MCXN947 at 150 MHz.

3.1 Key Systems Observations

- **NPU Memory-Bus Isolation:** Concurrent execution of NPU vision inference (2,973.04 μs) alongside CPU-bound PQC math produced an execution delta of +0.09 μs (+0.003%), confirming complete hardware bus isolation between the eIQ Neutron NPU DMA engine and the main Cortex-M33 bus.

Table 1: Bare-metal Post-Quantum Primitive Metrics on NXP FRDM-MCXN947 (150 MHz).

Algorithm	Operation	Mean Cycles	Time (ms)	Stack HWM (B)	Code Text (B)	Data (B)
ML-KEM-512	Keypair	2,978,432	19.86	8,960	41,648	40,928
ML-KEM-512	Encapsulate	3,503,918	23.36	11,592	41,648	40,928
ML-KEM-512	Decapsulate	4,342,471	28.95	12,344	41,648	40,928
ML-KEM-768	Keypair	4,893,951	32.63	13,056	41,432	42,400
ML-KEM-768	Encapsulate	5,692,464	37.95	16,200	41,432	42,400
ML-KEM-768	Decapsulate	6,809,160	45.39	17,272	41,432	42,400
ML-KEM-1024	Keypair	7,644,704	50.96	18,176	41,944	44,032
ML-KEM-1024	Encapsulate	8,582,539	57.22	21,832	41,944	44,032
ML-KEM-1024	Decapsulate	10,008,989	66.73	23,384	41,944	44,032
ML-DSA-87	Sign (Median)	18,790,358	125.27	28,117	44,000	42,100
ML-DSA-87	Sign (Worst-Case)	90,540,000	603.60	28,117	44,000	42,100

- **Algorithmic Rejection Sampling Fat-Tail:** ML-DSA signing latency displays a worst-to-median ratio of 4.8× to 8.1× (up to 603.6 ms). Because signature retry counts per seed are byte-for-byte identical across ARM Cortex-M33 and Xtensa LX7 silicon, this jitter is proven to be purely algorithmic (inherent to Fiat-Shamir rejection sampling) rather than architectural.

4 Classical vs. Post-Quantum Handshake Migration Study

To evaluate the migration impact from classical TLS 1.2 to PQC, an equivalent authentication handshake phase was evaluated:

$$\text{Handshake Phase} = 1 \times \text{Key Exchange} + 1 \times \text{Digital Signature} + 2 \times \text{Signature Verifications}$$

Table 2: Handshake Phase Migration Comparison across Target Microcontrollers.

Target Board	Cryptographic Suite	Total Latency (ms)	Active Energy (mJ)
NXP MCXN947	Classical (ECDH + ECDSA P-256)	166.10 ms	1.36 mJ
NXP MCXN947	PQC Level 1 (ML-KEM-512 + ML-DSA-44)	151.00 ms (**−9.1%**)	0.88 mJ (**−35%**)
NXP MCXN947	PQC Level 3 (ML-KEM-768 + ML-DSA-65)	250.60 ms (+50.9%)	1.48 mJ (+8.8%)
ESP32-S3	Classical (ECDH + ECDSA P-256)	495.20 ms	48.30 mJ
ESP32-S3	PQC Level 1 (ML-KEM-512 + ML-DSA-44)	88.80 ms (**−82.1%**)	8.60 mJ (**−82%**)
ESP32-S3	PQC Level 3 (ML-KEM-768 + ML-DSA-65)	134.20 ms (**−72.9%**)	13.10 mJ (**−73%**)

Takeaway: On the ESP32-S3, PQC Level 1 is **82.1% faster** than classical ECDH/ECDSA because ML-DSA signature verification is computationally lightweight compared to software-based ECDSA point verification.

5 DTLS Connection ID (RFC 9146) Energy Amortization Model

A 24-hour battery deployment model was built using raw 10 kHz PPK2 current measurements on a standard CR2032 coin cell (225 mAh capacity). It compares running a full post-quantum handshake every wakeup against resuming session states via DTLS Connection IDs.

Table 3: 24-Hour Battery Energy Profiles under Full Handshakes vs. DTLS CID Resumption.

Board Target	Wakeups / Day	Full Handshake (mJ)	DTLS CID Resumption (mJ)	Energy Savings
MCXN947 (PQC 768+87)	24 wakeups/day	1,340.91 mJ	1,297.90 mJ	3.21%
MCXN947 (PQC 768+87)	288 wakeups/day	1,834.92 mJ	1,298.23 mJ	29.25%
MCXN947 (PQC 768+87)	2,880 wakeups/day	6,685.23 mJ	1,301.50 mJ	**80.53%**
ESP32-S3 (PQC 768+87)	24 wakeups/day	1,735.13 mJ	1,315.61 mJ	24.18%
ESP32-S3 (PQC 768+87)	288 wakeups/day	6,565.57 mJ	1,330.69 mJ	79.73%
ESP32-S3 (PQC 768+87)	2,880 wakeups/day	53,991.65 mJ	1,478.69 mJ	**97.26%**

Takeaway: At dense duty cycles (2,880 wakeups/day), DTLS CID session resumption extends coin-cell battery lifespan on the ESP32-S3 from **88 days to 1,740 days (**19× **extension)**.

6 Single-Core Sliced Cooperative Scheduling and Obfuscation

To execute post-quantum handshakes alongside a 30 FPS (33.3 ms frame budget) real-time control loop on a single CPU core:

- **Cooperative Handshake Slicing:** Handshakes are divided into non-blocking operational chunks executed during available frame timing slack. This reduces real-time control deadline misses from **10.0% down to 0.0%** while reducing CPU execution footprint by 14–15%.
- **Side-Channel Power Obfuscation:** Inserting probabilistic dummy-state calculations ($p = 0.3$) into idle timing slack masks 10 kHz power-envelope activity leakage, degrading an attacker's side-channel classification accuracy from 100% down to **75.5%**.

7 Codebase Organization and Execution Commands

All experiment source code, hardware drivers, and evaluation scripts are structured within the repository as follows:

- `codebase/projects/securemark_tls_mcxn947/`: Bare-metal MCXN947 SecureMark-TLS C driver, mbedTLS wrappers, and `migration_compare.py`.
- `codebase/projects/securemark_tls_esp32s3/`: ESP32-S3 ESP-IDF SecureMark-TLS C driver and `cross_platform_comp`
- `codebase/mlkem_bench/` & `mldsa_bench_nxp/`: Bare-metal cycle counter drivers (`dwt.h`), stack painting scripts, and raw PQC datasets.
- `codebase/knowledge/`: Raw CSV benchmark logs (`dtls_cid_daily_energy.csv`, `mlkem_perf_memory_summary.csv`, `mldsa87_sign_raw.csv`).

7.1 Commands to Run and Verify Results

From the repository root directory, execute:

1. **MCXN947 Classical vs. PQC Migration Analysis:**

   ```
   python codebase/projects/securemark_tls_mcxn947/migration_compare.py
   ```

   Outputs: Generates $\texttt{results/MIGRATION}_{C}OMPARISON.mdandcomparativemigrationmetrics.$
2. **ESP32-S3 Cross-Platform Migration Analysis:**

   ```
   python codebase/projects/securemark_tls_esp32s3/cross_platform_compare.py
   ```

   Outputs: Generates $\texttt{results/CROSS}_{P}LATFORM_{M}IGRATION.md.$
3. **DTLS Connection ID 24-Hour Battery Longevity Model:**

   ```
   python codebase/projects/securemark_tls_mcxn947/dtls_cid_model.py
   ```

   Outputs: Generates daily energy savings CSVs and hourly power curves under $\texttt{results/dtls}_{c}id/.$
4. **ESP32-S3 SecureMark-TLS Build & Flash:**

   ```
   bash codebase/projects/securemark_tls_esp32s3/scripts/build.sh build
   bash codebase/projects/securemark_tls_esp32s3/scripts/build.sh -p COM17 flash
   ```

   Outputs: Serial console logs confirming a validated 1,895 SecureMark Score (14/14 CRC passes).